

- Supervised learning models define a mapping from one or more inputs to one or more outputs.
- The model itself is a mathematical function that describe a family of relationships.
 - ↳ The parameters specify the exact relationship
 - ↳ Passing an input to receive a prediction is called inference.
- A learning algorithm takes the training data (input, output) pairs; manipulates the parameters until the inputs predict the output as accurately as possible.
- We then see whether the learned model generalized to perform on unseen data.

2.1: Supervised Learning Overview:

- We want to build a model that takes an input x ; outputs prediction y .
- Assume both x, y are vectors of the same size; ordered in the same way } Tabular Data
- To make a prediction we need a model $f(\cdot)$ that takes in x ; outputs y :
$$y = f(x)$$
 } Equation of a fixed form.
represents a family of equations
- The model has parameters, θ , that determine the exact relationship between x & y :

... parameters, θ , that determine the exact relationship between x & y :

$$y = f[x, \theta]$$

- When training a model we try to find parameters θ that make sensible output predictions from the training data.
- The training data consists of I pairs of input & output examples $\{x_i, y_i\}$.
- We quantify the mismatch of the current model with a loss L . Since only the parameters are updated during the process, we can treat loss as a function of the parameters:

$L[\theta]$ $\rightarrow$ quantifies how poorly the model predicts the output from their corresponding inputs for parameters θ

- Since we aim to find parameters that minimize the loss function:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} [L[\theta]]$$

- If the loss is small after training, then we found parameters $\hat{\theta}$ that accurately predict training outputs y_i from training inputs x_i .

- After training, we must assess on a test dataset of unseen data to see how the model generalizes.

2.2: Linear Regression Example:

- Suppose a 1D linear regression model:

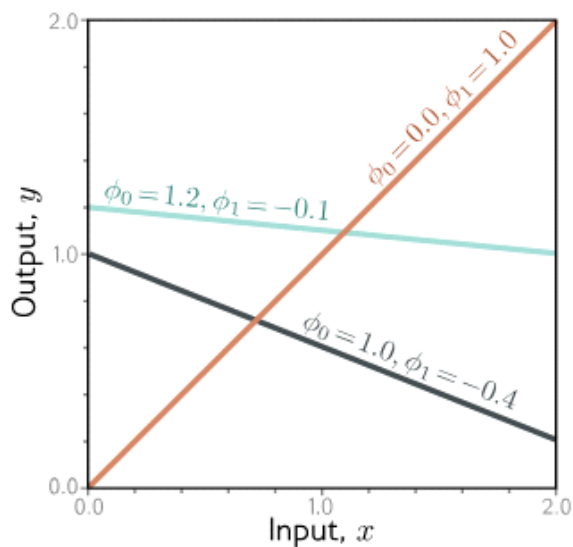
$$y = f[x, \phi]$$

$$= \phi_0 + \phi_1 x \quad \rightarrow \text{A straight line}$$

- The Model has two parameters:

$$\phi = [\phi_0, \phi_1]$$

- The choice of parameters determine the exact relationship,



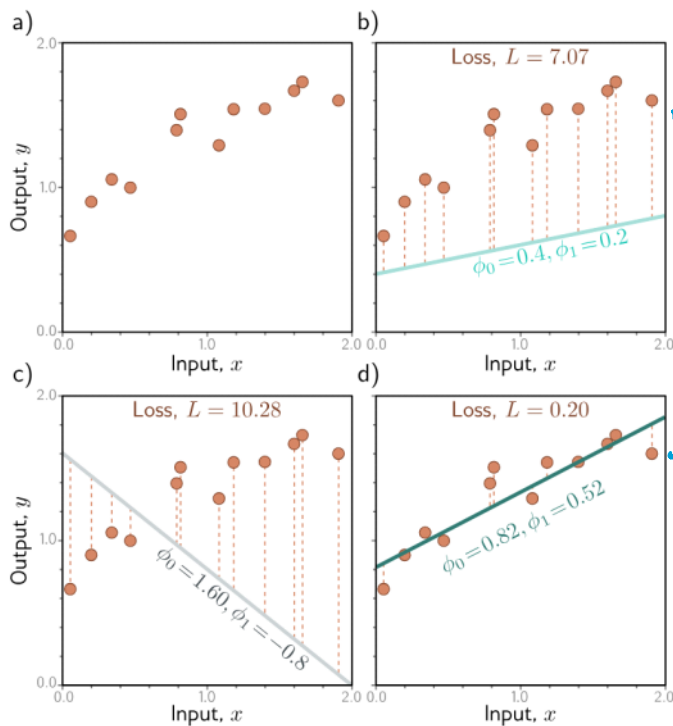
- To find the optimal parameters we must minimize a loss function

- To find the optimal parameters, we must minimize a loss function.
 ↳ Since we want the mismatch between y_i & $f(x_i, \theta)$, we can quantify the total training error/loss as the sum of squares of $f(x_i, \theta) - y_i$ for all I training pairs:

$$L[\theta] = \sum_{i=1}^I (f(x_i, \theta) - y_i)^2$$

$$= \sum_{i=1}^I (\theta_0 + \theta_1 x_i - y_i)^2 \quad \text{Least Squares loss}$$

- * The squaring operation means we don't care about the direction of the error.
- * When $L[\theta]$ is low, the model is a better fit.



↳ When θ is not optimal, $L[\theta]$ is high.

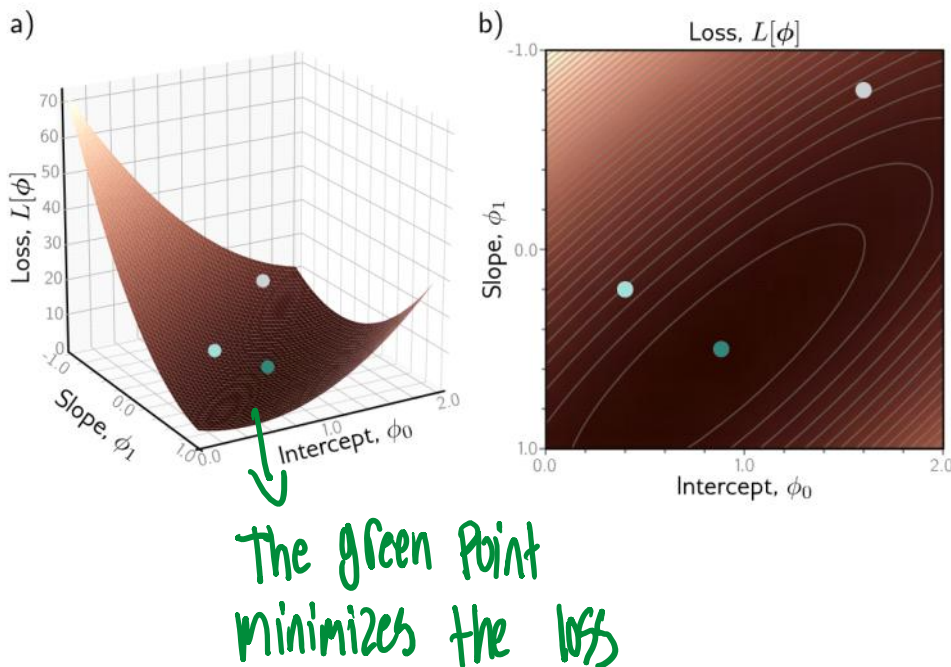
↳ When θ is near optimal, $L[\theta]$ is low.

- We want to find the optimal parameters that minimize the loss function, so:

- WE WANT TO FIND THE OPTIMAL PARAMETERS THAT MINIMIZE THE LOSS FUNCTION, SO:

$$\begin{aligned}\hat{\phi} &= \underset{\phi}{\operatorname{argmin}} [L[\phi]] \\ &= \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \right]\end{aligned}$$

- Since there are only two parameters, we can visualize the loss function as a surface:



- Training the model is done using gradient descent.
 - ↳ Choose initial parameters randomly
 - ↳ Compute the gradient of the surface at the current parameters
 - ↳ Take a step in the direction that is steepest downhill.
 - ↳ Repeat steps 2-4 until the gradient is flat & we can improve

↳ Repeat steps 2-4 until the gradient is flat & we can improve no further.

• Testing the model is done by computing the loss on a separate dataset.

↳ Generalization partly depends on how representative & complete the training data is.

↳ It also depends on how expressive the model is.

↳ When the model is too simple to capture the relationship, we underfit.

↳ When the model is so complicated that it describes the statistical peculiarities of the training data, not observed outside of that dataset, we overfit.

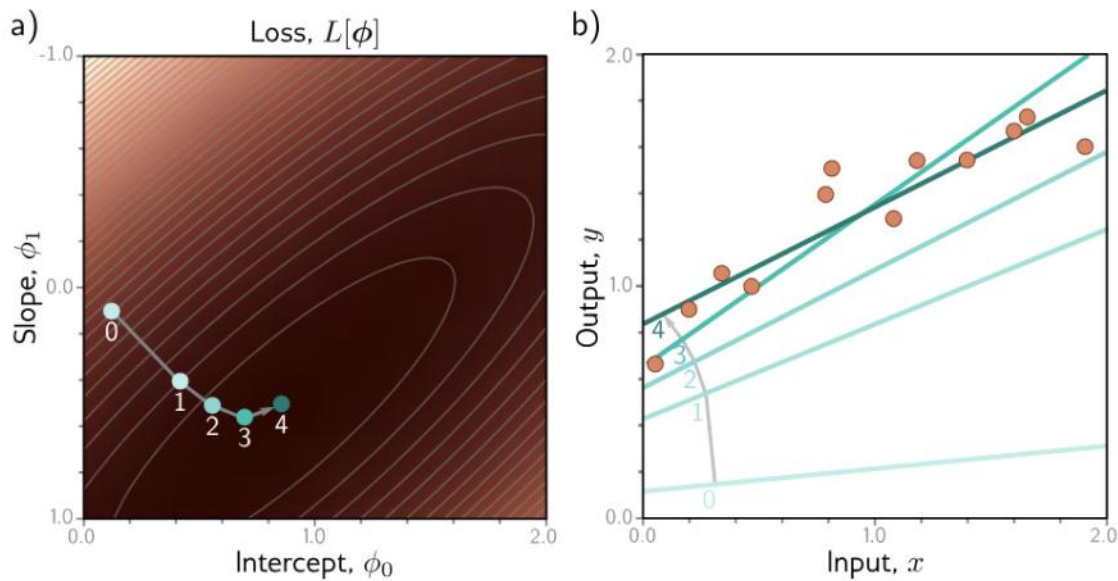


Figure 2.4 Linear regression training. The goal is to find the y-intercept and slope parameters that correspond to the smallest loss. a) Iterative training algorithms initialize the parameters randomly and then improve them by “walking downhill” until no further improvement can be made. Here, we start at position 0 and move a certain distance downhill (perpendicular to the contours) to position 1. Then we re-calculate the downhill direction and move to position 2. Eventually, we reach the minimum of the function (position 4). b) Each position 0–4 from panel (a) corresponds to a different y-intercept and slope and so represents a different line. As the loss decreases, the lines fit the data more closely. ([Interactive figure](#))